Overview

Build a Python program that randomly mutates a DNA sequence, translates the original and mutated versions, and reports what the mutation did to the protein. You will then run thousands of random mutations and find out which outcomes are most common.

You already wrote a DNA-to-protein translator in an earlier class. Reuse it here: the translator is the engine, and this project is about breaking DNA and measuring the damage.

You are going to be given a starter file, mutation_simulator_starter.py, with a codon table, a working translate() function and empty function stubs for you to fill in.

Biology background

A mutation is a change in a DNA sequence. Your code should handle the following three kinds:

Mutation type	What changes	Example
Substitution	One base is swapped for another	ATG**G**CC → ATG**A**CC
Insertion	One extra base is added	ATGGCC → ATG**T**GCC
Deletion	One base is removed	ATGGCC → ATGCC

Your program must then classify what the mutation did to the protein:

Effect	What happens to the protein	Example
Silent	Nothing; the new codon codes for the same amino acid	GCC → GCA, both Ala
Missense	One amino acid is replaced	GCC → GAC, Ala becomes Asp
Nonsense	A codon becomes a stop codon, so the protein is cut short	AAA → TAA, Lys becomes Stop
Frameshift	An insertion or deletion shifts every codon after it	Usually scrambles the rest of the protein

Example of a frameshift:

None

- Original: ATG GCC TTT AAA TAG -> Met-Ala-Phe-Lys
- Insert T at position 3

Mutated: ATG TGC CTT TAA ATA G -> Met-Cys-Leu

One inserted base changed every codon after it and created an early stop codon.

Roles

Each group member owns one part of the program. The parts connect through the function names below, so keep the names and return values exactly as written. In a group of three, the Simulator also takes on the Reporter's work.

Role 1: Mutator

Write the functions that change a DNA sequence. Each one picks a random position.

- substitute(seq) returns (new_seq, position). The new base must differ from the old one.
- insert(seq) returns (new_seq, position) with one random base added.
- delete(seq) returns (new_seq, position) with one base removed.
- random_mutation(seq) picks one of the three at random and returns (new_seq, kind, position), where kind is "substitution", "insertion" or "deletion".

Skills needed: string slicing, the random module.

Role 2: Classifier

Write the function that decides what a mutation did.

- classify(original_protein, mutated_protein, kind) returns "silent", "missense", "nonsense" or "frameshift".
- Insertions and deletions are always "frameshift".
- For substitutions, compare the two protein lists: identical means silent, a shorter mutated protein means nonsense, anything else means missense.

Skills needed: comparing lists, if/elif logic.

Role 3: Simulator

Write the loop that runs many experiments.

- run_simulation(seq, trials, mode="any") mutates the original sequence trials times, translates and classifies each result, and returns a dictionary of counts, for example {"silent": 241, "missense": 690, ...}.
- When mode is "substitution", only substitutions are used. When it is "any", use random_mutation().

Skills needed: loops, dictionaries, calling other people's functions.

Role 4: Reporter

Present the results.

- print_report(counts) prints each category with its count and percentage of all trials.
- plot_results(counts) draws a bar chart with matplotlib and gives it a title and axis labels.
- Write one demo run that shows a single mutation step by step: original sequence and protein, what changed, mutated sequence and protein, and the classification.

Skills needed: string formatting, matplotlib.